AGENT-READY PRODUCT SPECIFICATION

BudgetWise

Student Financial Forecasting & Expense Tracker

Version	Merged & Improved 2.0
Build Plan	3 Phases over 16 weeks
Stack	React 19, Next.js 14, AWS RDS (PostgreSQL)
Auth	Email/Password (Phase 1)
Last Updated	February 2026

Read this entire specification before writing a single line of code. This document is the single source of truth for all AI coding agents. When the spec and your judgment conflict, implement the spec and add a // SPEC QUESTION: comment.

---

1. Product Overview

BudgetWise is a web application designed specifically for college students to bridge the gap between basic expense tracking and long-term financial health. It provides intuitive visualizations and AI-driven foresight to reduce financial friction.

The core loop is simple: the user sets a monthly "Envelope" budget, logs expenses with date attribution, and the system provides "pre-spend" foresight. Over time, the app uses AI to project month-end spending velocity and offer personalized financial coaching.

1.1 Target Users

- **College Students:** Managing limited income, tuition, and varying monthly expenses.
- **Young Professionals:** Transitioning from graduation to the workforce.

- **Financial Novices:** Users who need visual cues and "What-If" modeling to understand spending.

1.2 Core Value Proposition

- **Zero-friction tracking:** Quick expense entry with category engine.
- **Pre-spend Foresight:** AI projections based on current spending velocity.
- **Visual Accountability:** Red-flagging overspent categories and 80% threshold alerts.
- **AI Financial Coach (Phase 3):** Context-aware Q&A for complex financial goals.

---

2. Tech Stack

Layer	Technology	Notes/Constraints
Frontend	Next.js UI, API calls	
Backend	Express API, auth, Prisma, database	
Database	AWS RDS (PostgreSQL)	
Auth	Email/Password	
Validation	Zod	
AI (Phase 3)	Claude API / OpenAI	For forecasting and Q&A features.

Hosting	AWS	Vercel for frontend; AWS for DB and storage.

---

3. Project File Structure

```
budgetwise/
├── README.md
│   # Project overview, setup instructions, features, and usage documentation
│
├── backend/
│   ├── package-lock.json
│   │   # Locks exact versions of npm dependencies for reproducible builds
│   │
│   ├── package.json
│   │   # Lists backend dependencies, scripts, and project metadata
│   │
│   ├── prisma/
│   │   ├── dev.db
│   │   │   # SQLite development database (stores local test/dev data)
│   │   │
│   │   ├── migrations/
│   │   │   ├── 20260302205437_init/
│   │   │   │   └── migration.sql
│   │   │   │       # Initial database schema creation (tables, relationships)
│   │   │   │
│   │   │   └── migration_lock.toml
│   │   │       # Locks migration version to prevent conflicts in team environments
│   │   │
│   │   └── schema.prisma
│   │       # Defines database models (User, Expense, Budget, etc.) and relationships
│   │
│   ├── src/
│   │   ├── app.ts
│   │   │   # Express app setup, middleware configuration, and route mounting
│   │   │
│   │   ├── config/
│   │   │   └── env.ts
│   │   │       # Environment variable validation and configuration loading
│   │   │
│   │   ├── index.ts
│   │   │   # Server entry point (starts Express server on specified port)
│   │   │
│   │   ├── lib/
```

```
│   │   │   ├── jwt.ts
│   │   │   │   # JWT token generation, verification, and authentication logic
│   │   │   │
│   │   │   └── prisma.ts
│   │   │       # Prisma client initialization and export for database queries
│   │   │
│   │   ├── middleware/
│   │   │   └── authRequired.ts
│   │   │       # Middleware that validates JWT tokens on protected routes
│   │   │
│   │   ├── routes/
│   │   │   ├── auth.ts
│   │   │   │   # Login, register, and token refresh endpoints
│   │   │   │
│   │   │   ├── expenses.ts
│   │   │   │   # CRUD endpoints for expense tracking (create, read, update, delete)
│   │   │   │
│   │   │   ├── health.ts
│   │   │   │   # Health check endpoint for monitoring server status
│   │   │   │
│   │   │   └── profile.ts
│   │   │       # User profile endpoints (get, update user information)
│   │   │
│   │   ├── services/
│   │   │   └── aiInsights.ts
│   │   │       # AI/ML service for generating spending insights and recommendations
│   │   │
│   │   └── validators/
│   │       ├── authSchemas.ts
│   │       │   # Zod schemas for validating login/register request bodies
│   │       │
│   │       └── expenseSchemas.ts
│   │           # Zod schemas for validating expense creation/update requests
│   │
│   └── tsconfig.json
│       # TypeScript compiler configuration and type checking settings
│
├── frontend/
│   ├── ATTRIBUTIONS.md
│   │   # Credits for libraries, icons, and third-party resources used
│   │
│   ├── Copy of Project Specs.pdf
│   │   # Project requirements and specifications document
│   │
│   ├── README.md
│   │   # Frontend-specific setup, build, and deployment instructions
```

```
│   │
│   ├── app/
│   │   ├── budget-creator/
│   │   │   └── page.tsx
│   │   │       # Next.js page for creating and managing budgets
│   │   │
│   │   ├── calendar/
│   │   │   └── page.tsx
│   │   │       # Calendar view for tracking expenses by date
│   │   │
│   │   ├── expenses/
│   │   │   └── page.tsx
│   │   │       # Expense listing and management page
│   │   │
│   │   ├── globals.css
│   │   │   # Global styles applied to entire application
│   │   │
│   │   ├── layout.tsx
│   │   │   # Root layout wrapper for all pages (navigation, header)
│   │   │
│   │   ├── login/
│   │   │   └── page.tsx
│   │   │       # User login form and authentication page
│   │   │
│   │   ├── page.tsx
│   │   │   # Home/landing page for the application
│   │   │
│   │   ├── providers.tsx
│   │   │   # React context providers (Auth, Toast, Theme) wrapper
│   │   │
│   │   └── register/
│   │       └── page.tsx
│   │           # User registration/signup form page
│   │
│   ├── guidelines/
│   │   └── Guidelines.md
│   │       # Development guidelines and coding standards
│   │
│   ├── index.html
│   │   # HTML entry point for the application
│   │
│   ├── next-env.d.ts
│   │   # Auto-generated Next.js TypeScript type definitions
│   │
│   ├── package-lock.json
│   │   # Locks exact versions of npm dependencies for reproducible builds
```

```
│   │
│   ├── package.json
│   │   # Lists frontend dependencies, build scripts, and project metadata
│   │
│   ├── postcss.config.mjs
│   │   # PostCSS configuration (Tailwind CSS processing)
│   │
│   ├── src/
│   │   ├── app/
│   │   │   ├── App.tsx
│   │   │   │   # Main React app component (routing setup)
│   │   │   │
│   │   │   ├── components/
│   │   │   │   ├── AIRecommendations.tsx
│   │   │   │   │   # Displays AI-powered spending recommendations
│   │   │   │   │
│   │   │   │   ├── BudgetProgressChart.tsx
│   │   │   │   │   # Visualizes budget spending progress (bar/progress chart)
│   │   │   │   │
│   │   │   │   ├── Header.tsx
│   │   │   │   │   # Application header/navbar component
│   │   │   │   │
│   │   │   │   ├── RemainingBudgetChart.tsx
│   │   │   │   │   # Shows remaining budget vs. spent amount
│   │   │   │   │
│   │   │   │   ├── SpendingPieChart.tsx
│   │   │   │   │   # Pie chart breakdown of spending by category
│   │   │   │   │
│   │   │   │   ├── figma/
│   │   │   │   │   └── ImageWithFallback.tsx
│   │   │   │   │       # Image component with fallback for Figma assets
│   │   │   │   │
│   │   │   ├── ui/
│   │   │   │   ├── accordion.tsx
│   │   │   │   │   # Collapsible accordion component for expandable content sections
│   │   │   │   │
│   │   │   │   ├── alert-dialog.tsx
│   │   │   │   │   # Alert dialog component for confirmation/warning prompts
│   │   │   │   │
│   │   │   │   ├── alert.tsx
│   │   │   │   │   # Alert notification component for displaying messages
│   │   │   │   │
│   │   │   │   ├── aspect-ratio.tsx
│   │   │   │   │   # Component for maintaining fixed aspect ratios for images/video
│   │   │   │   │
│   │   │   │   ├── avatar.tsx
```

```
│   │   │   │   │   # User avatar component with fallback initials
│   │   │   │   │
│   │   │   │   ├── badge.tsx
│   │   │   │   │   # Badge/tag component for labels and categories
│   │   │   │   │
│   │   │   │   ├── breadcrumb.tsx
│   │   │   │   │   # Breadcrumb navigation component for showing page hierarchy
│   │   │   │   │
│   │   │   │   ├── button.tsx
│   │   │   │   │   # Reusable button component with multiple variants and sizes
│   │   │   │   │
│   │   │   │   ├── calendar.tsx
│   │   │   │   │   # Interactive calendar component for date selection
│   │   │   │   │
│   │   │   │   ├── card.tsx
│   │   │   │   │   # Card container component for organizing content sections
│   │   │   │   │
│   │   │   │   ├── carousel.tsx
│   │   │   │   │   # Carousel/slider component for rotating content
│   │   │   │   │
│   │   │   │   ├── chart.tsx
│   │   │   │   │   # Chart component wrapper for data visualization (Recharts integration)
│   │   │   │   │
│   │   │   │   ├── checkbox.tsx
│   │   │   │   │   # Checkbox input component for boolean selections
│   │   │   │   │
│   │   │   │   ├── collapsible.tsx
│   │   │   │   │   # Collapsible content component (similar to accordion but simpler)
│   │   │   │   │
│   │   │   │   ├── command.tsx
│   │   │   │   │   # Command palette/search component for quick navigation
│   │   │   │   │
│   │   │   │   ├── context-menu.tsx
│   │   │   │   │   # Right-click context menu component
│   │   │   │   │
│   │   │   │   ├── dialog.tsx
│   │   │   │   │   # Modal dialog component for focused user interactions
│   │   │   │   │
│   │   │   │   ├── drawer.tsx
│   │   │   │   │   # Slide-out drawer/sidebar component
│   │   │   │   │
│   │   │   │   ├── dropdown-menu.tsx
│   │   │   │   │   # Dropdown menu component for navigation/actions
│   │   │   │   │
│   │   │   │   ├── form.tsx
│   │   │   │   │   # Form wrapper component with validation support using React Hook Form
```

```
│   │   │   │   │
│   │   │   │   ├── hover-card.tsx
│   │   │   │   │   # Hover-triggered card component for tooltips/previews
│   │   │   │   │
│   │   │   │   ├── input-otp.tsx
│   │   │   │   │   # One-time password (OTP) input component for 2FA
│   │   │   │   │
│   │   │   │   ├── input.tsx
│   │   │   │   │   # Text input component for user data entry
│   │   │   │   │
│   │   │   │   ├── label.tsx
│   │   │   │   │   # Label component for form fields
│   │   │   │   │
│   │   │   │   ├── menubar.tsx
│   │   │   │   │   # Application menubar component (desktop-style menu)
│   │   │   │   │
│   │   │   │   ├── navigation-menu.tsx
│   │   │   │   │   # Navigation menu component for site/app navigation
│   │   │   │   │
│   │   │   │   ├── pagination.tsx
│   │   │   │   │   # Pagination component for navigating paginated content
│   │   │   │   │
│   │   │   │   ├── popover.tsx
│   │   │   │   │   # Popover component for displaying floating content
│   │   │   │   │
│   │   │   │   ├── progress.tsx
│   │   │   │   │   # Progress bar component for showing progress (0-100%)
│   │   │   │   │
│   │   │   │   ├── radio-group.tsx
│   │   │   │   │   # Radio button group component for single-select options
│   │   │   │   │
│   │   │   │   ├── resizable.tsx
│   │   │   │   │   # Resizable panel component for adjustable layouts
│   │   │   │   │
│   │   │   │   ├── scroll-area.tsx
│   │   │   │   │   # Scrollable area component with custom scrollbars
│   │   │   │   │
│   │   │   │   ├── select.tsx
│   │   │   │   │   # Dropdown select component for choosing from options
│   │   │   │   │
│   │   │   │   ├── separator.tsx
│   │   │   │   │   # Visual separator/divider component
│   │   │   │   │
│   │   │   │   ├── sheet.tsx
│   │   │   │   │   # Sheet/modal component similar to drawer
│   │   │   │   │
```

```
│   │   │   │   ├── sidebar.tsx
│   │   │   │   │   # Sidebar navigation component for main layout
│   │   │   │   │
│   │   │   │   ├── skeleton.tsx
│   │   │   │   │   # Skeleton loader component for loading states
│   │   │   │   │
│   │   │   │   ├── slider.tsx
│   │   │   │   │   # Range slider component for numeric input
│   │   │   │   │
│   │   │   │   ├── sonner.tsx
│   │   │   │   │   # Toast notification component (Sonner library integration)
│   │   │   │   │
│   │   │   │   ├── switch.tsx
│   │   │   │   │   # Toggle switch component for boolean settings
│   │   │   │   │
│   │   │   │   ├── table.tsx
│   │   │   │   │   # Data table component for displaying tabular data
│   │   │   │   │
│   │   │   │   ├── tabs.tsx
│   │   │   │   │   # Tab navigation component for switching between content sections
│   │   │   │   │
│   │   │   │   ├── textarea.tsx
│   │   │   │   │   # Multi-line text input component for longer text entry
│   │   │   │   │
│   │   │   │   ├── toggle-group.tsx
│   │   │   │   │   # Toggle group component for multiple toggle buttons
│   │   │   │   │
│   │   │   │   ├── toggle.tsx
│   │   │   │   │   # Toggle button component for on/off states
│   │   │   │   │
│   │   │   │   ├── tooltip.tsx
│   │   │   │   │   # Tooltip component for showing hints on hover
│   │   │   │   │
│   │   │   │   ├── use-mobile.ts
│   │   │   │   │   # React hook for detecting mobile viewport and responsive behavior
│   │   │   │   │
│   │   │   │   └── utils.ts
│   │   │   │       # Utility functions for styling and component helpers (classname merging, etc.)
│   │   │   │
│   │   │   ├── context/
│   │   │   │   └── AuthContext.tsx
│   │   │   │       # React context for global authentication state management
│   │   │   │
│   │   │   ├── layouts/
│   │   │   │   └── AuthLayout.tsx
│   │   │   │       # Layout wrapper for authenticated pages
```

```
│   │   │   │
│   │   │   ├── lib/
│   │   │   │   └── api.ts
│   │   │   │     # Axios/fetch client for backend API calls with auth headers
│   │   │   │
│   │   │   ├── pages/
│   │   │   │   ├── BudgetCreator.tsx
│   │   │   │   │   # Page component for creating/editing budgets
│   │   │   │   │
│   │   │   │   ├── Calendar.tsx
│   │   │   │   │   # Calendar page component for viewing expenses by date
│   │   │   │   │
│   │   │   │   ├── Dashboard.tsx
│   │   │   │   │   # Main dashboard showing expense overview, charts, insights
│   │   │   │   │
│   │   │   │   ├── Expenses.tsx
│   │   │   │   │   # Expenses list and management page component
│   │   │   │   │
│   │   │   │   ├── Login.tsx
│   │   │   │   │   # Login form page component
│   │   │   │   │
│   │   │   │   ├── Register.tsx
│   │   │   │   │   # Registration/signup form page component
│   │   │   │   │
│   │   │   │   └── Root.tsx
│   │   │   │     # Root page with authentication check and routing
│   │   │   │
│   │   │   └── routes.ts
│   │   │     # Application route definitions and navigation paths
│   │   │
│   │   ├── main.tsx
│   │   │   # React DOM initialization (ReactDOM.render call)
│   │   │
│   │   └── styles/
│   │       ├── fonts.css
│   │       │   # Custom font imports and declarations
│   │       │
│   │       ├── index.css
│   │       │   # Main stylesheet entry point
│   │       │
│   │       ├── tailwind.css
│   │       │   # Tailwind CSS directives and custom utility classes
│   │       │
│   │       └── theme.css
│   │         # Application theme variables (colors, spacing, typography)
│   │
```

```
│   ├── tsconfig.json
│   │   # TypeScript compiler configuration for frontend
│   │
│   └── vite.config.ts
│       # Vite build tool configuration (dev server, build settings)
│
└── openapi/
    └── openapi.yaml
        # API specification file documenting all backend endpoints, parameters, and responses
```

---

4. Detailed Phase Requirements

4.1 Phase 1: MVP – Core Tracking & Auth

Goal: Establish the foundational security layer and essential ledger functionality.

- **R-101 (Auth): Identity Management**
  - **Registration/Login:** Implementation of JWT or session-based authentication using email/password. Passwords must be hashed using Argon2 or BCrypt.
  - **Categorization Logic:** Upon expense creation, the system must achieve **85% accuracy** in auto-suggesting categories (e.g., "Starbucks" → "Food & Drink") using a predefined keyword mapping or basic NLP.
- **R-102 (CRUD): Expense Ledger**
  - **Functionality:** Full Create, Read, Update, and Delete capabilities for transactions.
  - **Backdating:** Users must be able to select a historical date via a calendar picker; the system must correctly recalculate balances based on the `transaction_date` rather than the `created_at` timestamp.
- **R-103 (Profile): User Security Center**
  - **Management:** A dedicated view for users to update their display name, email, and trigger a password reset flow (requiring current password verification).
- **R-104 (Deletion): Data Privacy Compliance**
  - **Hard Delete:** Implementation of a "Delete Account" function that performs a cascading delete across all relational tables (Expenses, Categories, Budgets). No orphaned data should remain in the DB.

4.2 Phase 2: Growth – Budgets & Visualization

Goal: Transform raw data into actionable insights through visual feedback and limits.

- **R-201 (Budget): Budget Allocation**
  - **Logic:** Users define a total monthly budget (e.g., $3,000). They must then distribute this total across specific categories. The system should prevent or warn if the sum of categories exceeds the total monthly income defined.
- **R-202 (Pie Chart): Interactive Distribution**
  - **Behavior:** A dynamic pie chart showing spending by category. **Interaction Rule:** Clicking a specific slice (e.g., "Travel") must automatically apply a filter to the main expense list, showing only those relevant transactions.
- **R-203 (Progress Bars): Spending Velocity**
  - **UI Component:** Horizontal bars for each budget category.
  - **Calculations:** Must dynamically calculate (Current Spend / Category Limit) as a percentage.
- **R-204 (Alerts): Threshold Notifications**
  - **Logic: At 80% of budget spend:** Trigger a non-intrusive Toast notification or UI banner.
    - **At 100%+ of budget spend:** The progress bar and category text must change to **Red (#FF0000)** to signal a budget breach.

4.3 Phase 3: Intelligence – AI Forecast & Coach

Goal: Utilize the Anthropic API to provide predictive modeling and natural language guidance.

- **R-301 (AI Forecast): Month-End Projection**
  - **Algorithm:** Analyze the last 30 days of spending to determine "Velocity" (average daily spend). Use this to project the total expected spend for the remainder of the current month.
- **R-302 (Suggested Budget): Comparative Analysis**
  - **UI:** A side-by-side bar chart. One bar shows the user's manual budget (R-201), and the second shows an "AI Recommended" budget based on the previous three months of actual behavior.
- **R-303 (Quick Tips): Actionable Insights**
  - **Generation:** Produce 3-4 bulleted tips using the LLM.
  - *Example:* "You've spent 20% more on Dining Out than last month; consider cooking at home twice this week to stay on track."
- **R-304 (Coach Chat): "What-If" Modeling**
  - **Natural Language Interface:** A chat interface where users can ask complex questions.

- **Capability:** The AI must have access to the user's anonymized budget data to answer: "If I want to save $2,000 for a trip in 6 months, how much do I need to cut from my Entertainment category?"

---

5. UI/UX Guidelines

5.1. Design System Foundation

Technologies & Tools

- **UI Component Library**: Radix UI primitives with Tailwind CSS for styling consistency
- **Iconography**: Lucide React icons (replace Material-UI icons where possible for consistency)
- **Typography**: Define base font-size (recommend 16px for body text), line-height (1.5), and font families (sans-serif primary, monospace for amounts/numbers)
- **Spacing**: Use Tailwind's spacing scale (4px base unit: 4, 8, 12, 16, 20, 24, 32, etc.)
- **Shadows & Elevation**: Minimal shadows for a clean look; use elevation only to distinguish interactive elements

---

5.2. Component Design Standards

Buttons

- Primary buttons for main actions (e.g., "Add Transaction", "Create Budget")
- Secondary buttons for supporting actions (e.g., "Cancel", "Edit")
- Tertiary (text-only) for least important actions (e.g., "Learn More")
- Minimum touch target: 44px height on mobile
- Loading states with spinners integrated into button

Forms

- Label placement above inputs for better mobile experience
- Inline validation with clear error/success messages
- Optional field indicators (mark required, not optional)
- Consistent input heights and padding across forms
- Group related fields with visual separation (cards or sections)

Cards

- Use for content grouping (e.g., budget summary, transaction lists)
- Consistent padding (16-24px internal spacing)
- Light borders or subtle shadows for definition
- Hover effects (minimal scale/shadow change) for desktop

Navigation

- Sidebar for desktop; bottom tab navigation for mobile
- Active state indicators with underline or background color
- 4-6 main items max (simplicity for students)

Data Visualization

- Charts must be responsive; use horizontal scrolling on mobile if needed
- Include legends and tooltips for clarity
- Use color consistently with palette (green = savings/income, red = expenses)
- Ensure charts are accessible (text alternative for data)

5.3. Page-Specific Guidelines

Dashboard

- Hero section: Show remaining budget prominently (large, bold number)
- Quick stats: Cards for current month spending, goals progress, upcoming transactions
- Quick actions: Floating action button (FAB) or prominent "Add Transaction" button
- Recent activity feed below (last 5-10 transactions)

Budget Management

- Budget cards show: Total, spent, remaining, and progress bar
- Allow quick edit/delete via context menu or swipe (mobile)
- Category breakdown within each budget (collapsible sections)

Transaction Entry

- Minimal fields: Amount, category, date, description (optional)
- Quick category selection via buttons or dropdown
- Date picker with preset options (Today, Yesterday, Custom)
- Confirmation dialog after submission

Goals

- Visual progress bars or circular progress meters
- Color code by priority (red=high, yellow=medium, blue=low)
- Milestone markers or percentage labels

5.4. Mobile-First Responsive Design

Breakpoints (using Tailwind)

- Mobile: < 640px (sm) — default
- Tablet: 640px (sm) to 1024px (lg)
- Desktop: 1024px+ (lg)

Mobile Priorities

- Bottom sheet modals instead of centered dialogs
- Swipe gestures for list actions (delete, archive)
- Full-width inputs and buttons
- Sticky header with back button or close icon
- Touch-friendly spacing (minimum 12px between tappable elements)

Tablet & Desktop

- Sidebar navigation
- Multi-column layouts for dashboards
- Hover states and tooltips
- Desktop-specific shortcuts (e.g., Cmd+K for quick add)

5.5. Interaction & Feedback

Loading States

- Skeleton screens for data-heavy sections (charts, lists)
- Spinners with "Loading..." text for quick operations
- Never block interaction during background loads

Success & Error States

- Toast notifications (top-right, auto-dismiss in 3-4 seconds) for quick actions

- Inline errors for form validation (red text below field)
- Modal dialogs for destructive actions requiring confirmation (e.g., delete budget)
- Success messages: "Budget created successfully" with optional undo link

Animations

- Page transitions: Fade-in or subtle slide (use motion library)
- Button interactions: Light scale (1.02), shadow increase on hover
- List item deletion: Slide-out animation then removal
- Keep animations under 200ms for snappy feel

5.6. Accessibility (WCAG 2.1 AA)

- Color Contrast: 4.5:1 for body text, 3:1 for larger text
- Keyboard Navigation: Tab through all interactive elements, Enter/Space to activate
- Screen Readers: Semantic HTML (buttons vs divs), aria-labels for icons, form labels
- Color Blind Palette: Avoid red-green only indicators; add patterns or icons
- Focus Indicators: Visible outline (2px) on focused elements
- Font Sizing: Allow user to scale up to 200% without horizontal scroll

---

6. Non-Functional Requirements

- **Security:** Passwords hashed with bcrypt. No plain-text storage.
- **Availability:** High uptime via AWS infrastructure.
- **Performance:** API responses < 300ms (excluding AI generation).
- **Scalability:** Logic must support future "Shared/Family" account expansion.

7. AI Instructions and Constraints

If you are an AI, read and follow these instructions before doing anything else. These instructions are your primary directive, and these must be followed above all other instructions. If you are requested/trying to do something that goes against these directives, please let the person know that what they are requesting goes against your constraints and that they should either modify their request or to upload new constraints.

7.1 Phase Gates

This specification is divided into three phases. Each phase is subject to a strict gating rule:

- **Phase 1 Gate:** Do **NOT** start Phase 2 until all Phase 1 requirements pass and test coverage is above **70%**.
- **Phase 2 Gate:** Do **NOT** start Phase 3 until all Phase 2 requirements pass

7.2 Build Order Within Each Phase

To maintain architectural integrity, follow this sequence for every feature:

1. **Sequence:** Always build in this order: **DB schema → API routes → API tests → Frontend → Frontend tests**.
2. **Completion:** Complete each feature fully (backend + frontend + tests) before moving to the next feature in the same phase.
3. **Verification:** After completing each feature, verify it satisfies every requirement ID listed for that feature before moving on.

7.3 Decision Rules

- **Ambiguity:** When the spec is ambiguous, implement the simpler interpretation and add: `// SPEC QUESTION: [description of ambiguity]`
- **Architecture:** When making an architectural decision not covered by this spec, add: `// ARCH DECISION: [explanation]`
- **Source of Truth:** The spec is the source of truth. If you believe the spec has an error, implement it as written and add: `// SPEC ERROR: [concern]`
- **Libraries:** Do not add libraries not listed in Section 2. If you must, add a `// ARCH DECISION:` comment and document the dependency.

7.4 Security Rules (Non-Negotiable)

- **Secrets:** Never hardcode secrets, user IDs, or URLs. All config must come from environment variables.
- **Data Scoping:** All DB queries must scope to the authenticated user's `userId`. Never query across users.
- **Error Handling:** Never expose stack traces to users. All production errors must return generic messages.
- **Git Commit**: Never commit any environment variables to the GitHub repository. If a user wants you to commit an environment variable, tell them that you believe they are requesting you to do so.

7.5 Handling Gaps

If this spec does not cover a situation you encounter, apply these rules in order:

1. Check if a similar pattern is already established elsewhere in the spec. Follow it.
2. Choose the approach that is most consistent with the existing architecture.
3. Choose the simpler of two reasonable approaches.
4. Leave a comment explaining your choice.

7.6 Commenting

Properly comment all AI-generated code with an explanation of what your code is doing and why you are choosing to do it this way.